

BrokerVerse OOTB

Insurance broking platform by iorta TechNXT

Solution Architecture · Document 07 of 11

High Availability and Resiliency

Product	BrokerVerse OOTB
Document	07 High Availability and Resiliency
Version	1.1
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse OOTB Solution Architecture: 07 High Availability and Resiliency
Document ID	BV-SA-07
Version	1.1
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, deploy/, docker-compose.yml, .github/workflows, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue
1.1	29 September 2026	iorta TechNXT	Updated to the current system: placement journey, co-insurance, posting rules, period end and BIR tax, bank reconciliation, document numbering, commission rate matrix, single PDF engine, uploads and go-live imports, housekeeping, business time zone, broker roles, deployment on the BrokerVerse URL

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

- 1 Introduction..... 4
 - 1.1 Purpose and scope..... 4
- 2 Application properties that enable high availability..... 5
 - 2.1 Stateless API..... 5
 - 2.2 Health endpoints..... 5
 - 2.3 Start-up and graceful shutdown..... 5
 - 2.4 Scheduled jobs on several instances..... 5
 - 2.5 Transactions, retries and idempotency..... 6
- 3 Recommended highly available deployment..... 7
- 4 Single points of failure and mitigations..... 8
- 5 Known limitations..... 9
- 6 Related documents..... 10
- 7 Glossary..... 11

1 Introduction

1.1 Purpose and scope

This document describes how BrokerVerse OOTB stays available when components fail: the properties of the application that allow redundancy (stateless API, health endpoints, graceful shutdown, single execution of scheduled jobs and migrations, settings that follow changes on every instance), the recommended highly available deployment in AWS, the single points of failure and their mitigations, and the known limitations. Recovery objectives are in document 08, backups and restores in document 09.

Recommended / to be confirmed by the business and DevOps: `deploy/README.md` deploys the backend as a container behind an HTTPS load balancer, with PostgreSQL 16 and a persistent upload volume, and allows several instances. The topology in chapter 3 (two or more API instances across two Availability Zones, Multi-AZ RDS, EFS) is the recommended production form of that deployment; the number of instances, Multi-AZ and the file system choice are to be confirmed by DevOps.

2 Application properties that enable high availability

2.1 Stateless API

The API keeps no user session in memory. Every request carries its access token, and all state is in PostgreSQL (records, refresh tokens, token versions, job runs, outbox) or in the file store (`UPLOAD_DIR`). Any instance can serve any request, so instances can be added, removed or replaced behind a load balancer without sticky sessions. Two things are held per process: the settings cache, which checks the table version every 5 seconds and so follows changes made through other instances, and the rate-limit counters (chapter 5).

2.2 Health endpoints

Endpoint	Use	Behaviour (<code>backend/src/lib/health.js</code>)
GET <code>/api/health</code>	Readiness: load balancer target health, Docker HEALTHCHECK	200 { status: "ok", ready: true, database: { reachable, latencyMs }, pendingMigrations: 0, uptimeSeconds } when start-up (migrations and seed) has finished, PostgreSQL answers <code>SELECT 1</code> within <code>HEALTH_DB_TIMEOUT_MS</code> (2,000 ms) and no migration is pending; otherwise 503 with status <code>starting</code> or <code>unavailable</code> . Also 503 as soon as shutdown begins, so traffic drains.
GET <code>/api/health/live</code>	Liveness: container restart probe	200 while the process serves HTTP; no database check, so a database outage does not cause restart loops
GET <code>/api/version</code>	Monitoring	Name, version, commit (<code>GIT_COMMIT</code>), start time, uptime, Node version, environment, database reachability, pending migrations

The health endpoints need no sign-in, are not rate limited and are not written to the access log.

2.3 Start-up and graceful shutdown

1. The server checks the production configuration and exits with a clear log line if it is unsafe.
2. It marks itself not ready and takes the migration advisory lock. If another instance holds it, it waits (up to `MIGRATION_LOCK_TIMEOUT_MS`, 10 minutes by default) and then finds nothing left to apply. It applies pending migrations and the idempotent seed, starts HTTP and the scheduler, and marks itself ready.
3. On `SIGTERM` or `SIGINT` (task stop, rolling deployment, scale-in) it marks itself not ready (readiness 503), stops the scheduler, stops accepting new connections, lets requests in flight finish, closes idle keep-alive connections and the database pool, and exits. A forced exit happens after `SHUTDOWN_TIMEOUT_MS` (25 s by default).

Recommended / to be confirmed by the business and DevOps: Set the container stop timeout to 30 s (above the 25 s shutdown timeout) and the load balancer deregistration delay to 30 s, so requests in flight complete during deployments.

2.4 Scheduled jobs on several instances

Every job run takes a PostgreSQL session advisory lock keyed on the job code (`pg_try_advisory_lock`) on a dedicated connection; an instance that cannot take the lock skips the run. A scheduled run is also skipped if another instance already started a scheduled run of the same job in the same minute. The scheduler can

therefore run on all instances (`SCHEDULER_ENABLED=true`) without double execution, and the jobs keep running as long as one instance with the scheduler is up. Advisory locks are released automatically when the holding connection or instance dies. Every instance reads the cron expressions in the business time zone (Asia/Manila) and reloads the schedules within 30 seconds of a change made through any instance.

2.5 Transactions, retries and idempotency

Aspect	Behaviour
Atomic business operations	Multi-table operations (policy issue with bill, co-insurance split, journals, collection item and commission; receipts and applications; vouchers; debit notes; remittance decisions; close runs) run in one database transaction (<code>withTransaction</code>). A failure rolls back everything, including the document number, so a retry does not leave half-created documents.
Concurrency guards	Rows are locked <code>FOR UPDATE</code> where a double action would do harm (converting a quotation, issuing a policy, taking a document number); unique indexes prevent duplicates (one open renewal per policy, one pending approval per entity, one active close run per period, one lead co-insurer, unique document numbers, one bank statement per file hash).
Reruns	A month-end close run executed again first reverses its own earlier journals; the go-live opening balance load for the same date replaces the earlier load.
Client retries	The front end retries a request once after refreshing an expired access token. Other failed requests are not retried automatically; the user repeats the action. POST requests carry no idempotency keys, so a save that timed out but succeeded can be repeated by the user; the uniqueness rules above prevent the most harmful duplicates.
E-mail	Queued in the database and sent by the outbox job with up to 5 attempts; an SMTP outage delays e-mail but does not fail a business transaction.
Renewal notice queue	<code>job_queue</code> items are retried by the <code>renewal-queue</code> job; stale items are picked up again.
Database failover	During an RDS Multi-AZ failover (typically 60 to 120 s) open connections fail; requests in that window return 500 or 503 and the pool reconnects afterwards. Readiness answers 503 while the database is unreachable, but the liveness probe keeps the containers running.

3 Recommended highly available deployment

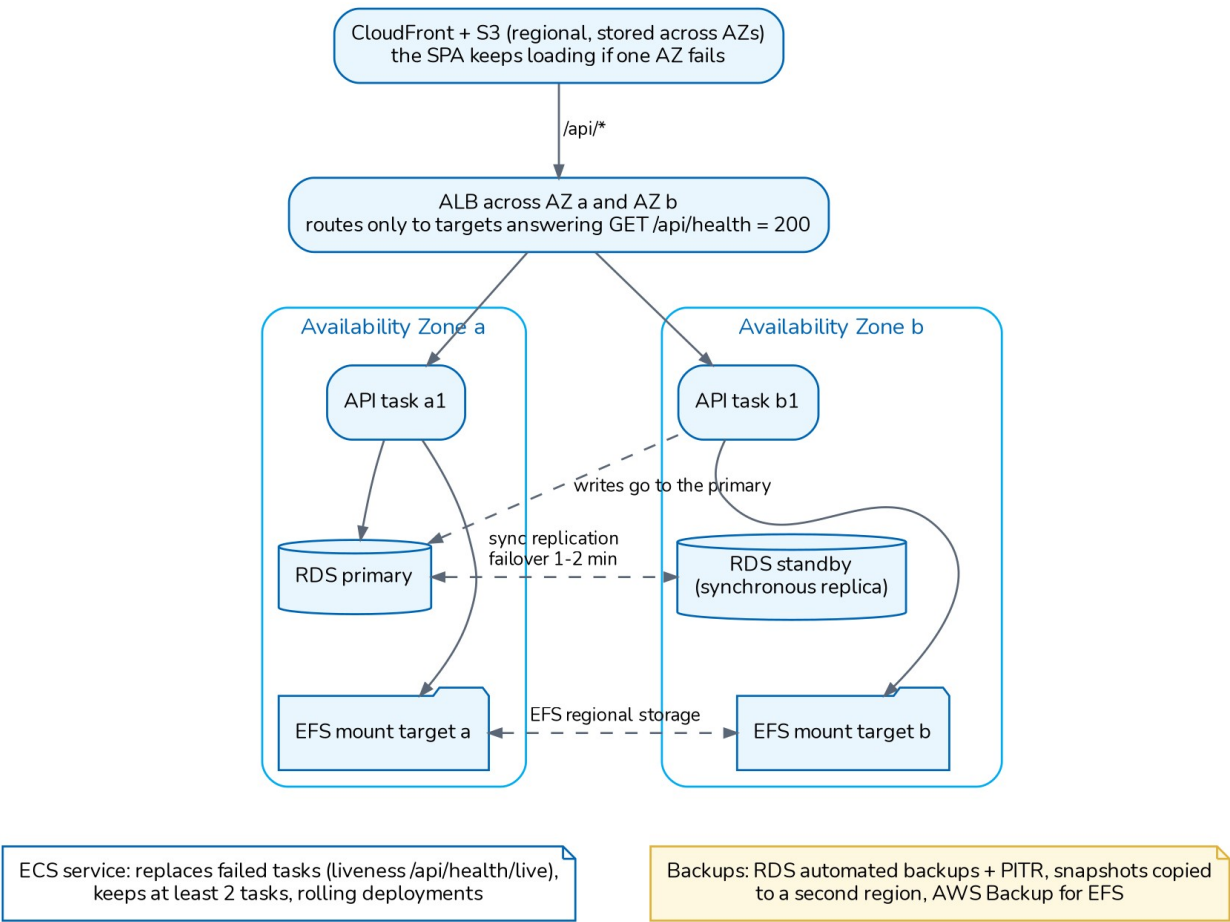


Figure 1. High-availability topology and failure domains (recommended)

Tier	Redundancy	Behaviour on failure
DNS and edge	DNS of the BrokerVerse URL, CloudFront (global edge), optional WAF	Managed by AWS; no single instance
Front end	S3 (regional, stores objects across Availability Zones) with versioning	The SPA stays available during API outages; screens show errors until the API returns
Load balancer	Application Load Balancer across two Availability Zones	Managed; routes only to healthy targets (<code>/api/health</code>)
API	At least two containers spread over two Availability Zones; rolling deployment with minimum healthy 100%, maximum 200%	A failed container is replaced automatically (liveness); if a zone fails, the containers in the other zone keep serving and replacements start there
Database	RDS PostgreSQL Multi-AZ (synchronous standby)	Automatic failover to the standby in the other zone, same endpoint name; the application reconnects
File store	EFS (regional storage class, mount targets in each zone)	Files remain available from the other zone
Secrets	Secrets Manager or SSM (regional)	Read at container start
E-mail	Office 365	Outbox retries; e-mail is delayed, not lost

4 Single points of failure and mitigations

Single point of failure	In the baseline	Mitigation
Database	One PostgreSQL instance unless Multi-AZ is chosen	RDS Multi-AZ in production; point-in-time recovery and snapshots (document 09)
File store	A persistent volume mounted into the container; a local volume serves one instance only	EFS mounted by all instances; AWS Backup
Single API instance	The checklist starts with one instance; Docker Compose runs one	At least two instances in two zones once the first start has applied the migrations
Migrations at start	A failed migration stops every starting instance; the release is not rolled back automatically	Run one instance first for releases with migrations; test migrations on a UAT copy of production; keep the previous image and a pre-deployment snapshot for rollback
Secrets	Losing <code>DATA_ENCRYPTION_KEY</code> makes two-factor secrets unreadable; changing <code>JWT_SECRET</code> signs everyone out	Secret store with versioning; sealed copy of the keys kept with the backups (document 09)
Region	Single region ap-southeast-1	Cross-region snapshot and file backup copies; documented regional recovery (document 08)
E-mail provider	One Office 365 mailbox in <code>SMTP_URL</code>	Outbox retries; another provider can be set by changing <code>SMTP_URL</code> and the sender setting
Front-end pipeline	Deployment by GitHub Actions to one bucket	S3 versioning allows a rollback to the previous build; the workflow can be re-run on a previous commit

5 Known limitations

Limitation	Impact and recommendation
Per-instance rate limiting	Sign-in and API rate limits are kept in memory per process. With N instances the effective limit is N times the configured value, and counters reset when an instance restarts. Recommended: move the counters to a shared store (PostgreSQL or ElastiCache Redis) or enforce limits with AWS WAF rate-based rules.
Long-running requests	Large report files, bulk prints and imports run inside the web request on the event loop; meanwhile the instance serves other requests slowly (document 06).
Local file storage API	Files are on a shared file system, not in S3; EFS gives multi-zone durability. Replacing the storage functions with S3 would remove the file system dependency (later change).
No automatic retry of failed jobs	A failed job run is recorded (<code>job_runs.status = failed</code>) and logged at error level; it runs again at its next schedule. There is no immediate retry; alerting is set up from the log (document 11).

6 Related documents

This document is one of eleven that together form the BrokerVerse OOTB Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency (this document)	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

7 Glossary

Terms and abbreviations used in this document.

Term	Meaning
Advisory lock	PostgreSQL application-level lock (pg_try_advisory_lock, pg_advisory_lock) used so a job runs on one API instance at a time and only one instance applies migrations.
API	Application programming interface; here the BrokerVerse REST API under /api served by the Node.js backend.
AZ	Availability Zone: an isolated data-centre group inside an AWS region.
CloudFront	AWS content delivery network that serves the single-page application and can route /api to the backend.
Co-insurance	Placement of one risk with several insurers, one of them the lead, each taking a share; premium, commission and claims are split by share.
Cron	Time-based schedule expression (minute hour day month weekday) used by the scheduled jobs.
EFS	Elastic File System (AWS): shared, multi-AZ network file system proposed for the upload directory.
Housekeeping	Daily job that deletes operational rows (job runs, sent e-mails, sign-in history, expired tokens, read notifications) after the retention days in System Settings.
JWT	JSON Web Token: signed token carrying the user id, roles, permissions and token version.
Migration	Numbered SQL file in backend/src/db/migrations applied once, in order, on API start.
Month-end close	Run (MEC-) that posts the accruals, recurring journals and valuations of a period, checks the checklist and, once approved, closes the period.
Multi-AZ	Deployment with a synchronous standby in a second Availability Zone and automatic failover (RDS).
Office 365	Microsoft 365 mail service; BrokerVerse sends e-mail through smtp.office365.com with the mailbox in SMTP_URL.
RDS	Amazon Relational Database Service, the managed PostgreSQL target.
Receivable / bill	Premium bill (INV-) raised when a policy, endorsement or renewal is issued.
Remittance	Payment of collected premium, net of commission, from the broker to the insurer.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
Seed	Idempotent data load run on every API start: reference data always, sample data only when SEED_SAMPLE_DATA is on.
SMTP	Simple Mail Transfer Protocol; the API sends e-mail through the server in SMTP_URL.
SPA	Single-page application: the React front end served as static files.
UAT	User acceptance testing environment.
WAF	Web Application Firewall.